Решение тестового задания на исследовательскую программу ФКН

Примечание: Замеры были сделаны в режиме эмуляции архитектуры x86_64 через docker на MacBook m1 pro (см. **Окружение** ниже). Также некоторые цифры могут не соответствовать в приложенных выводах ClangBuildAnalyzer, так как перед отправкой произвелось перетестирование

Анализ и выводы после запуска ClangBuildAnalyzer:

1. Время уходит во frontend. 37.9 из 54.3 s CPU — это ≈70% против 30% на backend. При -O3 backend заметен, но не он определяет стоимость сборки. Оптимизировать нужно frontend.
2. Файлы, которые тянут много заголовков, компилируются почти одинаковое время. Одни и те же заголовки разбираются заново в каждом TU и в каждой сборке.
3. Backend сконцентрирован в сгенерированном коде. Из 16.4 s backend 9.5 s (58%) приходится на четыре protobuf/gRPC-TU. Значит даже полное устранение повторного разбора заголовков не уберёт всю стоимость: codegen и оптимизация сгенерированного кода останутся.
4. Backend и линковка никуда не денутся (особенно при -O3 и LTO), поэтому оптимизировать имеет смысл именно frontend.

Гипотеза:

Если ~70% CPU уходит на повторный разбор общих заголовков, то предкомпилированные заголовки (PCH) для общих наборов userver и protobuf/gRPC сократят frontend, а вместе с ним и суммарную работу компилятора.

Альтернатива — перевод архитектуры на C++20 modules, но из-за ограниченной поддержки в компиляторах на этапе исследования выбран более практичный вариант — PCH.

Изменения:

В CMakeLists.txt добавлена опция RESEARCH_USE_PCH которая включает прекомпиляцию хедеров, которые занимаю большую часть времени (src/pch_userver.hpp и src/pch_proto.hpp), замеры производились по 5 раз на baseline и pch.

Важно отметить, что теперь будут учитываться не 10 компиляций, а 13, так как добавляется компиляция самих pch файлов.

Сами PCH стоят 8.8 с (3.11 + 3.12 + 2.53 с), так что чистый выигрыш frontend - 10.4 с (19.2 с экономии в TU минус стоимость PCH). По TU: `main.cpp` 6.56->4.04 с (-38.5%), `hello_postgres.cpp` 4.12 -> 2.22 (–46.5%), `hello_grpc.cpp` 3.79 -> 2.44 (-35.5%), `hello_service.usrv.pb.cpp` 5.08->3.18 (-37.3%), `hello_client.usrv.pb.cpp` 5.11 -> 3.27 (–36.1%), `hello.grpc.pb.cc` 3.69->1.57 (-57.4%), `hello.pb.cc` 2.21->0.69 (-68.9%). Маленькие `hello.cpp`, `hello_mongo.cpp` и `greeting.cpp` ускорились на 74.8-87.8%, потому что почти весь их frontend был чужими заголовками.

Метрика	Baseline (sec)	PCH (sec)	delta
frontend	37.9	27.5	–27.4%
backend	16.4	16.3	–0.6%
CPU total	54.3	43.8	–19.3%

Метрика	Baseline (sec)	PCH (sec)	delta
wall-clock	29.9	29.5	–1.34%

Что из этого следует? Гипотеза подтвердилась для стоимости компиляции: -27.4% frontend и -19.3% CPU при неизменном backend. Но wall-clock почти не изменился: медиана 29.9->29.5 с (-1.34%), в трёх прогонах из пяти PCH быстрее, один прогон (31.6 с) попал на всплеск нагрузки на хосте. Причина видна в фазах: на 4 ядрах параллелизм уже частично прячет лишнюю работу, PCH-сборки добавляют последовательный этап на критический путь, а 12-секундная линковка не меняется вовсе. Для этого небольшого сервиса PCH — оптимизация CPU/энергии и пропускной способности CI, а не wall-clock.

Проверка результата на более реальном проекте (https://github.com/oartemiy/internview_cxx/tree/pch-bench)*:

Метрика	Baseline (sec)	PCH (sec)	delta
frontend	58.8	29.0	-50.7%
backend	9.3	9.3	+0.0%
CPU total	68.1	38.3	-43.8%
wall-clock	193.76	186.04	-4.0%

* — Результаты были замерены нативно локально на хосте — Apple M1 Pro (10 ядер, 16 ГБ, MacOS)

Вывод:

В результате выполнения исследования, видно, что использование pch дает достаточно большой прирост, особенно на более реальном проекте, где прирост может доходить до 50%. Таким образом, выдвинутая гипотеза подтвердилась и показала свою ценность не только на шаблонном проекте, но и на более реальном проекте.

Окружение:

Ubuntu 22.04.5, Clang 18.1.8, CMake 4.4.4. хост — Apple M1 Pro (10 ядер, 16 ГБ, macOS), поэтому x86-64 образ работает через эмуляцию. Абсолютные секунды поэтому условны, но baseline и эксперимент сняты в одинаковых условиях, и сравнивать их между собой корректно.

Сборка выполнялась на Release режиме

Использование ИИ:

1. Написание скриптов, помогающих проверяющему без проблем быстро проверить результаты
2. Python скриптов, которые парсят результаты из файлов, сгенерированных ClangBuildAnalizer, и высчитывают проценты для отчета
3. Верификации результатов и поиск ошибок

Воспроизведение:

1. Рекомендуется использовать официальный docker-образ userver, см. задание
2. Клонировать проект (https://github.com/oartemiy/userver-research)
3. ./scripts/setup_env.sh

4. ./scripts/bootstrap_cba.sh
5. make measure